Guia da reunião

NEON aplicado à MLP int8 (784→64→10, MNIST)

Davi — Iniciação Científica, Engenharia Elétrica, UFMG
Orientador: Prof. Vitor Ângelo — 10/09/2026

Sumário

1 O que foi feito, em uma página **2**

2 Como rodar **3**

3 O modelo **3**

3.1 Dados . . . 3
3.2 Padronização (StandardScaler) . . . 3
3.3 Rede . . . 4
3.4 Por que ReLU . . . 4
3.5 Por que não há softmax na inferência . . . 4

4 Quantização e ponto fixo **5**

4.1 Ideia . . . 5
4.2 Escalas . . . 5
4.3 Como o C calcula sem nenhum float . . . 5

5 O kernel NEON **6**

6 Integração na MLP **7**

7 Como os resultados foram medidos e por quê **7**

7.1 Correção: comparação bit a bit . . . 7
7.2 Acurácia e matriz de confusão . . . 8
7.3 Tempo no orquestrador (x86, só indicativo) . . . 8
7.4 Tempo no benchmark NEON (método do professor) . . . 8

8 Resultados medidos **10**

8.1 MLP modular (x86 nativo, orquestrador) 10
8.2 Matriz de confusão do int8 (idêntica no NEON) 10
8.3 Kernel NEON isolado (aarch64 no QEMU) 10
8.4 MLP int8 com NEON (aarch64 no QEMU) 11
8.5 Código gerado (GCC 15, `make verificar-asm`) 11

9 Expectativas para a placa (hipóteses) 11

10 Propostas de medição (próximos passos) 12
10.1 Intervalo de confiança de Wilson (acurácia) 12
10.2 Teste de McNemar (comparar dois modelos) 12
10.3 Correção de Bonferroni 12
10.4 Ablação do recorte (análise preliminar) 13
10.5 Para o benchmark 13

11 Glossário de variáveis e arquivos 13

12 Perguntas prováveis 14

1 O que foi feito, em uma página

1. **MLP modular (já pronta):** inferência em C nas versões float32, int16 e int8, validada bit a bit contra o Python (`c_modular/`).
2. **NEON (já pronto):** produto escalar int8 com intrinsics, validado na placa nos meus experimentos anteriores (`origem/`).
3. **Trabalho novo:** apliquei o kernel NEON, sem alterar o corpo, às duas camadas da MLP int8, sem alterar a MLP (`c_neon/`), e validei a correção e a medição.

Resultados principais (medidos):
Acurácia: float32 0,9708; int16 0,9725; int8 escalar 0,9728; int8 NEON 0,9728.
NEON × escalar: logits idênticos nas 10.000 imagens; matriz de confusão idêntica.
Kernel isolado: 746.785 casos sem nenhuma diferença; reproduz o 576483 da placa.
Desempenho na Raspberry Pi 4: **ainda não medido** (Seção 9).

2 Como rodar

Requisitos (Linux ou WSL em PC x86):

```
sudo apt install -y gcc make gcc-aarch64-linux-gnu \
     binutils-aarch64-linux-gnu qemu-user
```

Tudo de uma vez (grava as saídas em `resultados/`):

```
bash rodar_tudo.sh
```

Ou etapa por etapa:

```
cd c_modular && make && ./orquestrador todos    # x86 nativo
cd ../c_neon && make                            # compila p/ ARM
make testar-kernel     # kernel NEON x escalar   (QEMU)
make validar-mlp       # MLP NEON x escalar x Python
make verificar-asm     # instruções NEON geradas
make ensaiar-bench     # benchmark curto (emulação)
```

Na placa (medição de desempenho):

```
./teste_dot_i8_neon ../modelo/exportado
./valida_mlp_int8_neon ../modelo/exportado
./bench_mlp_int8_neon 3 10000 3 21   # núcleo imagens runs trials
```

Por que QEMU? O NEON só existe em ARM. O QEMU executa o binário ARM no PC reproduzindo cada instrução exatamente: serve para provar a *correção*. O tempo no QEMU é o custo da emulação, não do Cortex-A72: *não* serve para desempenho.

3 O modelo

3.1 Dados

MNIST: imagens de dígitos 28×28 em tons de cinza (0 a 255), achatadas em 784 valores. Divisão canônica: 60.000 imagens de treino e 10.000 de teste. O teste nunca foi usado para ajustar nada.

3.2 Padronização (StandardScaler)

$$x_i = \frac{\text{pixel}_i - \text{média}_i}{\text{desvio}_i}$$

Média e desvio de cada pixel calculados no treino. Pixels sempre apagados (desvio zero) recebem desvio 1. Serve para colocar as entradas na mesma escala e facilitar o treino.

3.3 Rede

$$a_j = b_{0,j} + \sum_{i=1}^{784} W_{0,ji}\, x_i, \qquad h_j = \max(a_j, 0), \qquad z_k = b_{1,k} + \sum_{j=1}^{64} W_{1,kj}\, h_j$$

- 784 entradas → 64 neurônios ocultos → 10 saídas (uma por dígito).
- Treino: scikit-learn `MLPClassifier`: `activation="relu"`, `solver="adam"`, `alpha=1e-4` (regularização L2), `batch_size=128`, `learning_rate_init=1e-3`, `max_iter=30`, `early_stopping=True` (10% do treino para validação, para após 5 épocas sem melhora), `random_state=42`.

3.4 Por que ReLU

$\mathrm{ReLU}(a) = \max(a, 0)$.

- É a ativação padrão de redes densas: barata e sem saturação para valores positivos, o que ajuda o gradiente no treino.
- Em inteiros custa só uma comparação.
- Funciona com quantização: para escala $s > 0$, $\max(0, s\,A) = s \max(0, A)$. A ReLU pode ser aplicada direto no acumulador inteiro, sem voltar ao float.

3.5 Por que não há softmax na inferência

No treino, a saída usa softmax e entropia cruzada. Na inferência só importa a classe:

$$\mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

é crescente em z_k e tem o mesmo denominador para todas as classes. Logo $\arg\max_k \mathrm{softmax}(z)_k = \arg\max_k z_k$. A classe é o índice do maior logit; em empate, vence o menor índice.

4 Quantização e ponto fixo

4.1 Ideia

Cada valor real T vira um inteiro T_q e uma escala s:

$$T \approx s\,T_q, \qquad T_q = \mathrm{clip}\big(\mathrm{round}(T/s),\,-127,\,127\big).$$

- **Simétrica:** o zero real vira o inteiro 0, sem deslocamento (*zero point*). A multiplicação fica simples: $(s_a a_q)(s_b b_q) = s_a s_b\,(a_q b_q)$.
- **Por tensor:** uma escala por matriz; é a forma mais simples.
- **Faixa** $[-127, 127]$**:** simétrica; o -128 não é usado.

4.2 Escalas

Escala	Como é calculada	Valor
s_{W0}	$\max\|W_0\|/127$	0,0045042
s_{W1}	$\max\|W_1\|/127$	0,0050596
s_x	percentil 99,9 de $\|x\|$ em 5.000 imagens de treino / 127	0,063711
s_h	percentil 99,9 de $\|h\|$ em 5.000 imagens de treino / 127	0,13930

Por que percentil 99,9 nas ativações e não o máximo? Alguns pixels quase nunca acendem no treino e têm desvio muito pequeno; quando acendem, a entrada padronizada fica enorme (até 419,6 desvios no teste). Usar o máximo desperdiçaria quase toda a faixa do int8 com esses casos raros. O percentil recorta os extremos (limite de 8,09 desvios) e preserva a resolução dos valores comuns.

4.3 Como o C calcula sem nenhum float

1. **Entrada por tabela:** `x = MLP8_LUT[i*256 + pixel]`. A tabela (784×256) guarda padronização, quantização e recorte já calculados. Como cada pixel só tem 256 valores, é exata.
2. **Camada oculta:** `A = MLP8_B0[j] + soma(x * W0)`, em int32. O bias foi pré-escalado: $B_0 = \mathrm{round}(b_0/(s_x s_{W0}))$, para ficar na mesma unidade da soma.
3. **ReLU e requantização:**

   $$h = \min\big((\max(A, 0) \cdot M + 2^{N-1}) \gg N,\ 127\big)$$

   com $M/2^N \approx s_x s_{W0}/s_h$, $M = 1.132.510.016$ e $N = 39$. O termo 2^{N-1} faz arredondar em vez de truncar. O produto é feito em int64.

4. **Saída:** `Z = MLP8_B1[k] + soma(h * W1)`. Não precisa requantizar: a escala $s_h s_{W1}$ é positiva e igual nas 10 classes, então o argmax não muda.

Exemplo: pixel central (índice 406) aceso (255): média 130,267, desvio 113,727, logo $x = 1{,}0968$ desvios e $x/s_x = 17{,}21$, e a tabela guarda 17. Acumulador $A = 10.000$: $(10.000 \cdot M + 2^{38}) \gg 39 = 21$, contra o valor real $10.000 \cdot s_x s_{W0}/s_h = 20{,}60$.

Por que int16? Controle de precisão: mesmo método do int8, trocando 127 por 32767. Se o int8 estivesse errado, o int16 se afastaria dele. O acumulador int16 chega a $7{,}9 \times 10^9$ e por isso usa int64.

5 O kernel NEON

```
int32_t dot_i8_neon(const int8_t *a, const int8_t *b, int len)
{
  int32x4_t acc = vdupq_n_s32(0);
  for (int i = 0; i < len; i += 16) {
    int8x16_t va = vld1q_s8(a + i);
    int8x16_t vb = vld1q_s8(b + i);
    acc = vpadalq_s16(acc, vmull_s8(vget_low_s8(va),
                                    vget_low_s8(vb)));
    acc = vpadalq_s16(acc, vmull_s8(vget_high_s8(va),
                                    vget_high_s8(vb)));
  }
  int32x2_t s = vadd_s32(vget_low_s32(acc),
                         vget_high_s32(acc));
  return vget_lane_s32(vpadd_s32(s, s), 0);
}
```

Intrinsic	O que faz
`vld1q_s8`	carrega 16 bytes num registrador de 128 bits
`vget_low/high_s8`	separa as metades de 8 elementos
`vmull_s8`	multiplica 8 pares int8 e dá 8 resultados int16 (instrução `smull`)
`vpadalq_s16`	soma os int16 em pares, alarga para int32 e acumula nos 4 acumuladores (`sadalp`)
`vadd`, `vpadd`, `vget_lane`	somam os 4 acumuladores e devolvem um int32

- **Por que alargar:** $127 \times 127 = 16.129$ e $(-128)^2 = 16.384$ não cabem em int8, mas cabem em int16. A soma de 784 produtos cabe em int32.
- **16 elementos por iteração:** um registrador de 128 bits guarda 16 int8.
- **Restrição:** o tamanho deve ser múltiplo de 16 (não há tratamento de sobra). Na MLP, $784 = 16{\times}49$ e $64 = 16{\times}4$.
- **Cortex-A72 é ARMv8.0:** não tem `vdotq_s32` (extensão ARMv8.2). Por isso se usa `smull` com `sadalp`.

- **Por que manual:** no meu relatório anterior, o GCC vetorizava o laço escalar com 6 instruções por bloco (`smull` + `saddw`); o manual usa 4 (`smull` + `sadalp`).

6 Integração na MLP

- `mlp_int8_neon.c` repete o pipeline de `c_modular/mlp_int8.c` e troca só os dois laços de produto escalar:

```
A = MLP8_B0[j] + dot_i8_neon(x, &MLP8_W0[j*784], 784);
Z = MLP8_B1[k] + dot_i8_neon(h, &MLP8_W1[k*64], 64);
```

- **Pesos contíguos:** as matrizes estão guardadas com uma linha por neurônio (`W0[j*784+i]`), então cada neurônio já é um vetor contíguo.
- **Bias fora do kernel:** soma inteira, o resultado não depende da ordem.
- **Nada existente foi alterado:** o corpo do kernel é o mesmo de `homogeneidade_v2.c`, e `mlp_int8.c` tem o mesmo md5 (`e8476fde`).

7 Como os resultados foram medidos e por quê

7.1 Correção: comparação bit a bit

Como: a mesma imagem passa pela versão escalar e pela NEON, e cada valor é comparado exatamente: entrada quantizada, produto, produto + bias, ReLU, ativação h, logits e classe. Os logits também são comparados com a referência gerada em Python (`ref_int8.bin`).

Por quê:

- **Igualdade é o esperado:** a aritmética é inteira e sem estouro, então qualquer diferença, por menor que seja, é erro.
- **A acurácia sozinha esconde erros.** Exemplo com os meus números: float32 e int8 diferem em só 20 acertos (9708 contra 9728), mas tomam **47 decisões diferentes**. Duas versões com acurácia parecida podem classificar imagens diferentes.
- **Localiza a falha:** a comparação por estágio mostra exatamente onde uma divergência começa.

Kernel isolado: antes da MLP, o kernel é testado sozinho contra o escalar e contra a soma exata em 64 bits, em tamanhos múltiplos de 16, casos extremos, desalinhamento de memória, o vetor da placa (576483) e os vetores reais das duas camadas.

7.2 Acurácia e matriz de confusão

- **Acurácia:** acertos / 10.000.
- **Matriz de confusão:** célula [real][previsto] conta quantas imagens do dígito real foram classificadas como o previsto; a diagonal são os acertos. Mostra *quais* dígitos a rede confunde (ex.: 2 com 8, 4 com 9).
- **Precisão da classe k:** dos que a rede chamou de k, quantos eram k.
- **Revocação da classe k:** dos que eram k, quantos a rede reconheceu.
- **F1:** média harmônica de precisão e revocação.

Por quê: matriz idêntica entre NEON e escalar confirma, célula a célula, que as decisões são as mesmas.

7.3 Tempo no orquestrador (x86, só indicativo)

Como: `c_modular/orquestrador.c` cronometra cada inferência com `CLOCK_MONOTONIC` e reporta a média por imagem numa passagem pelas 10.000 imagens, sem fixar núcleo nem aquecer.

Por quê: o objetivo do orquestrador é validar. O tempo dá só uma ordem de grandeza em x86 e varia entre execuções (o float mediu 30 µs numa e 42 µs noutra). **Não vale para a placa.**

7.4 Tempo no benchmark NEON (método do professor)

`c_neon/bench_mlp_int8_neon.c` segue a metodologia de `origem/experimentos_neon_2/homogeneidade_v2.c`:

Decisão	Justificativa
Fixar a thread em um núcleo (`pthread_setaffinity_np`)	evita que o sistema migre o processo e perca a cache; no Orange Pi os núcleos são diferentes. No Pi 4 os núcleos se mostraram homogêneos (spread de 0,0% em `homogeneidade_v2`), mas fixar mantém as condições iguais
Carregar as imagens na RAM antes	leitura de arquivo é lenta e variável; não é inferência
Aquecer (1 passagem inicial e 1 inferência antes de cada bloco)	a primeira execução paga falhas de página, cache fria e a subida de frequência da CPU
`CLOCK_PROCESS_CPUTIME_ID`	conta só o tempo de CPU do processo; tempo em que outro programa usou a CPU não entra
Cronometrar `runs` passagens juntas e dividir	uma inferência dura microssegundos; medir blocos grandes dilui o custo do próprio relógio
Vários trials (padrão 21)	uma medição isolada pode pegar interrupção do sistema
Mediana (e média e mínimo)	a mediana ignora picos raros; média muito acima da mediana indica picos
Girar a ordem das variantes a cada trial	`ordem_cache.c` mostrou que medir logo depois de outra variante pode favorecer a segunda (razão frio/quente de até 1,46)
Mesmas imagens nas três variantes	comparação justa
Três variantes: escalar puro, autovetorizado e NEON	o GCC com `-O2` já vetoriza o escalar (confirmado no assembly); sem `-fno-tree-vectorize` estaríamos comparando NEON manual com NEON automático
Portão de correção e soma de verificação	só mede se as três variantes derem logits idênticos, e confere que o trabalho medido foi o mesmo
Registrar governador, frequência e temperatura	frequência e aquecimento mudam o tempo
Tempo por imagem; *speedup* = mediana do escalar / mediana do NEON	unidade comparável entre execuções

Limitações conhecidas desta medição:

- todos os trials rodam num único processo;
- o argmax e a soma de verificação entram no tempo (igual para as três variantes);
- o kernel NEON é chamado por função (74 chamadas por imagem), o que pesa só na variante NEON.

8 Resultados medidos

8.1 MLP modular (x86 nativo, orquestrador)

Versão	Acurácia	Bit a bit	Decisões ≠ float	Tempo/imagem*
float32	0,9708	sim	0	29,9 µs
int16	0,9725	sim	36	7,7 µs
int8	0,9728	sim	47	3,2 µs

*x86 no WSL, uma passagem; indicativo, não vale para a placa.

O mesmo código compilado para aarch64 e executado no QEMU também valida bit a bit.

8.2 Matriz de confusão do int8 (idêntica no NEON)

real\prev.	0	1	2	3	4	5	6	7	8	9
0	**968**	1	2	0	0	1	4	2	2	0
1	0	**1125**	3	0	0	1	2	1	3	0
2	3	2	**995**	7	4	0	3	4	13	1
3	1	0	3	**982**	0	7	1	6	5	5
4	0	0	9	1	**959**	0	4	1	3	5
5	3	0	1	13	3	**857**	4	0	9	2
6	3	3	4	1	5	5	**935**	0	2	0
7	1	4	14	3	4	0	0	**991**	3	8
8	3	2	2	4	4	3	3	3	**948**	2
9	3	4	1	5	12	1	0	5	10	**968**

Maiores confusões: 7→2 (14), 5→3 (13), 2→8 (13), 9→4 (12). A pior classe é o 8 (precisão 0,9499), e a melhor, o 1 (F1 0,9886). A média macro do F1 é 0,9726.

8.3 Kernel NEON isolado (aarch64 no QEMU)

Grupo	Casos	Diferenças
Vetor da placa (576483)	1	0
Tamanhos 16 a 4096 (múltiplos de 16)	5.120	0
Extremos e impulsos (784 e 64)	864	0
Desalinhamento de memória	800	0
Vetores reais, camada 784→64	640.000	0
Vetores reais, camada 64→10	100.000	0

8.4 MLP int8 com NEON (aarch64 no QEMU)

Comparação nas 10.000 imagens	Valores	Diferenças
Entrada quantizada	7.840.000	0
Produto, produto+bias, ReLU, h (cada)	640.000	0
Logits	100.000	0
Classe prevista	10.000	0
Logits NEON × escalar (c_modular)	100.000	0
Logits NEON × Python	100.000	0

Acurácia 0,9728 nas duas versões; 0 classificações divergentes.

8.5 Código gerado (GCC 15, `make verificar-asm`)

Objeto	Instruções NEON
escalar puro (`-fno-tree-vectorize`)	nenhuma
`mlp_int8.c` com `-O2`	2 `smull`, 2 `smull2`, 4 `saddw`, 4 `saddw2` (vetorizado pelo GCC)
kernel manual	2 `smull`, 2 `sadalp`

9 Expectativas para a placa (hipóteses)

Correção: expectativa forte. Mesmos logits bit a bit, acurácia 0,9728 e 0 divergências. A aritmética é inteira e o mesmo binário ARM já validou no QEMU. Primeiro passo na placa: rodar `teste_dot_i8_neon` e `valida_mlp_int8_neon`.

Desempenho: hipóteses a medir.

- **Kernel isolado, experimentos anteriores na placa:** `regimes_memoria` deu 5 a 6× (dados no L1D e no L2). `homogeneidade_v2` deu 2,2× (mediana). Nesse último os flags não foram registrados, e o escalar levou 0,45 ns por elemento, rápido demais para um laço sem vetorização: provavelmente comparou NEON manual com escalar já vetorizado.
- **Na MLP o ganho deve ser menor**, porque parte do tempo não é produto escalar: 784 leituras da tabela, 64 requantizações e 74 chamadas por imagem. Pela lei de Amdahl, com fração f do tempo no produto escalar e kernel S vezes mais rápido, speedup $= 1/((1-f) + f/S)$:

$f \setminus S$	1,5×	2,2×	5,5×
0,80	1,36	1,77	2,89
0,90	1,43	1,96	3,79
0,95	1,46	2,08	4,49

- **Faixas esperadas:** NEON contra escalar puro, de 3 a 4,5×; NEON contra autovetorizado, de 1,3 a 2×, ou menos se as chamadas de função pesarem.
- **Memória:** os pesos da camada 1 (49 KB) e a tabela (200 KB) cabem no L2 de 1 MB do Cortex-A72, regime em que o kernel manteve speedup alto.

10 Propostas de medição (próximos passos)

Não fazem parte dos resultados acima; são análises propostas para dar mais rigor.

10.1 Intervalo de confiança de Wilson (acurácia)

O que é: 10.000 imagens são uma amostra; se fossem outras 10.000, a acurácia mudaria um pouco. O intervalo de 95% dá a faixa plausível da acurácia verdadeira. O método de Wilson é o recomendado para proporções próximas de 1.
Com os meus números: int8 0,9728 com intervalo [0,9694; 0,9758], cerca de ±0,3 ponto percentual. Diferenças menores que isso entre dois modelos não podem ser afirmadas só olhando as acurácias.

10.2 Teste de McNemar (comparar dois modelos)

O que é: olha só as imagens em que os modelos discordam: b = só o modelo B acerta; c = só o A acerta. Se os dois fossem igualmente bons, cada discordância seria uma moeda justa. O teste diz a probabilidade p de um desequilíbrio igual ou maior acontecer por acaso.
Por que serve: os dois modelos são avaliados nas *mesmas* imagens, e comparar as acurácias separadamente ignora isso.
Com os meus números (o orquestrador já imprime): float × int8, 30 contra 10, $p = 0{,}0022$; int16 × int8, 7 contra 4, $p = 0{,}55$ (compatível com o acaso).

10.3 Correção de Bonferroni

O que é: com o limiar de 5%, cada teste tem 5% de chance de dar "significativo" por acaso. Fazendo muitos testes, isso se acumula. A correção divide o limiar pelo número de testes: com 10 testes, exige $p < 0{,}005$.

10.4 Ablação do recorte (análise preliminar)

Acrescentando uma transformação por vez ao modelo float: só o recorte da entrada muda a acurácia de forma relevante (0,9708 → 0,9725). O float recortado toma as mesmas decisões do int16. Isso sugere que o int8 acerta um pouco mais que o float por causa do recorte, e não da quantização. Precisa ser confirmado com mais sementes de treino.

10.5 Para o benchmark

- **Repetir o programa várias vezes (ex.: 5):** mede a variação entre execuções, e não só entre trials.
- **Intervalo da mediana por *bootstrap*:** reamostrar os trials para obter a faixa plausível da mediana e do speedup.
- **Medir também o kernel isolado** em 64, 784 e 16.384 elementos, para comparar com a MLP inteira e com os experimentos anteriores.
- **Testar entradas fora do MNIST** (ruído, imagens invertidas, deslocadas) na validação.
- **Kernel `inline`:** eliminar as 74 chamadas por imagem sem mudar o corpo.

11 Glossário de variáveis e arquivos

Nome	Significado
`x, a (A), h, z (Z)`	entrada, pré-ativação (acumulador), ativação após ReLU, logit
`s_x, s_W0, s_h, s_W1`	escalas de quantização (só existem no Python)
`MLP8_LUT[i*256+p]`	entrada i quantizada para o pixel de valor p
`MLP8_W0[j*784+i]`	peso int8 da entrada i para o neurônio j
`MLP8_B0[j]`	bias int32 do neurônio j (unidade $s_x s_{W0}$)
`MLP8_W1[k*64+j]`	peso int8 do neurônio j para a classe k
`MLP8_B1[k]`	bias int32 da classe k (unidade $s_h s_{W1}$)
`MLP8_M, MLP8_N`	$M/2^N \approx s_x s_{W0}/s_h$ (requantização)
`MLP8_LIM`	127, limite do int8
`MLPF_MEDIA, MLPF_ESCALA`	média e desvio do StandardScaler (float)
`teste_u8.bin`	10.000 registros: 1 byte de rótulo + 784 pixels
`ref_int8.bin`	logits int32 esperados, gerados em Python
`dot_i8_neon / dot_i8_ref`	produto escalar NEON / escalar
`mlp_int8_inferir`	inferência int8 escalar (c_modular)
`mlp_int8_neon_inferir`	inferência int8 com NEON (c_neon)
`runs, trials`	passagens por bloco cronometrado; repetições da medição

12 Perguntas prováveis

Usou ReLU? Sim, na camada oculta (`activation="relu"`). Na saída não há ativação na inferência: a classe é o argmax dos logits.

E a softmax? Só no treino (entropia cruzada). Não muda o argmax.

Por que int8 e simétrica? int8 é o menor tipo com multiplicação vetorizada eficiente em NEON (16 valores por registrador). Simétrica elimina o zero point e simplifica a multiplicação.

O que são M e N? Inteiros que substituem o fator real $s_x s_{W0}/s_h$: multiplicar por M e deslocar N bits equivale a multiplicar por 0,00206, sem float.

Por que a tabela de entrada? Evita 784 divisões em float por imagem; é exata porque o pixel só tem 256 valores.

Por que o int8 acerta mais que o float? A diferença é de 20 imagens. A análise preliminar indica que vem do recorte das entradas extremas, e não da quantização (Seção 10).

Como sei que o NEON está certo? Logits idênticos ao escalar e ao Python em todas as 10.000 imagens, em cada estágio, e o kernel testado sozinho em 746.785 casos.

Por que rodar no QEMU? Para provar correção sem a placa. O tempo no QEMU não vale.

Por que três variantes no benchmark? O GCC já vetoriza o escalar com `-O2`; o escalar puro exige `-fno-tree-vectorize`.

Por que mediana? É robusta a picos causados pelo sistema operacional.

Por que `CLOCK_PROCESS_CPUTIME_ID`? Mede o tempo de CPU do processo e ignora o tempo em que outro programa ocupou a CPU; é o método do professor.

E se o vetor não for múltiplo de 16? O kernel leria além do vetor. Na MLP não acontece (784 e 64).

O que falta? Validar e medir na Raspberry Pi 4 e depois otimizar (kernel `inline`).